

UNIT-3

Peer-to-Peer Systems – Introduction

♦ What is Peer-to-Peer (P2P)?

- **Definition:**

A distributed system architecture where each computer (peer) acts as both a **client** and a **server**, sharing resources directly without relying on a centralized server.

- **Key Idea:**

Every peer contributes resources (storage, bandwidth, processing power) and consumes resources from others.

♦ Characteristics of P2P Systems

1. **Decentralization**

- No single central server; peers communicate directly.

2. **Resource Sharing**

- Files, CPU cycles, storage, and bandwidth are shared among peers.

3. **Scalability**

- System can grow easily by adding more peers.

4. **Fault Tolerance**

- Failure of one/few peers does not crash the system.

5. **Dynamic Membership**

- Peers can **join** or **leave** at any time.

♦ Advantages of P2P

- Efficient resource utilization.

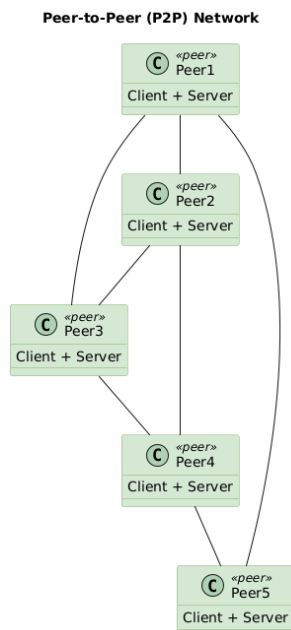
- Reduces bottlenecks compared to client-server systems.
- Cost-effective (no dedicated server).
- Good for **file sharing, content distribution, and collaborative applications**.

♦ Challenges in P2P

- **Security threats** (malicious peers, data tampering).
- **Free-riding** problem (peers using resources without sharing).
- **Search efficiency** (finding resources quickly).
- **Scalability issues** for large networks without proper overlay structure.

♦ Examples of P2P Systems

- **File sharing**: Napster, BitTorrent.
- **Distributed storage**: OceanStore.
- **Distributed computing**: SETI@home.
- **Blockchain & Cryptocurrencies**: Bitcoin, Ethereum.



Napster and Its Legacy

♦ Napster (1999)

- One of the **first popular P2P systems** for music file sharing.
- **Architecture:**
 - **Centralized directory server:** Stores the list of available songs and which peers have them.
 - **Peers (users):** Share MP3 files.
 - When a user searches a song → Query sent to central server → Server replies with peer list → Download happens **peer-to-peer**.

♦ Problems

- **Single Point of Failure** (central server).
- **Legal issues** (copyright infringement).
- **Scalability issues.**

♦ Legacy

- Inspired next-gen **decentralized P2P systems** like **Gnutella, BitTorrent, Kazaa**.
- Showed potential of **resource sharing & content distribution**.

Core idea

- A **hybrid P2P** system: peers store the **files**, but a **central index server** stores **who has which file**.
- Users searched the **central directory**; downloads happened **peer-to-peer**.

Architecture (roles)

- **Central directory server**

- Maintains: user logins, peer IP/port, list of file names each peer shares.
- Handles: search queries, referrals (returns a list of peers holding the file).
- **Peers (clients)**
 - On login: **register** shared file list with the server.
 - On search: **query** the server; receive a list of peers.
 - On download: **connect directly** to another peer to fetch the MP3.

Typical operation (conceptual)

1. **Register:** Peer → Server (list of filenames/metadata).
2. **Search:** Peer → Server (“song.mp3”).
3. **Referral:** Server → Peer (peer list {IP:port,...}).
4. **Download:** Peer ↔ Peer (direct TCP transfer).
5. **Updates:** Periodic heartbeats / re-register on reconnect.

Why it took off

- **Fast search** (central index).
- **Simple UX** (one place to ask).
- **Bandwidth scales** with number of users (downloads are peer-to-peer).

Limitations (important points)

- **Single point of failure/attack:** if the **directory** goes down, search stops.
- **Legal exposure:** central server “knew” about copyrighted content → easy target.
- **Scalability pressure:** the index must handle all queries/updates.

- **Incentives:** free-riding (some users download but don't share).
- **Churn handling:** peers join/leave; directory entries become stale.

Peer-to-Peer Middleware

Introduction

- **P2P Middleware:** Software layer that provides abstraction and support for building distributed applications on top of peer-to-peer systems.
- It hides complexity like:
 - Node discovery
 - Resource location
 - Communication between peers
 - Fault tolerance and replication

Functions of P2P Middleware

1. Peer Discovery & Management

- Keeps track of available peers.
- Handles peers joining/leaving dynamically.

2. Resource Location

- Maps resources (files, data, services) to peers.
- Uses distributed lookup (DHT – Distributed Hash Tables, flooding, or hybrid methods).

3. Communication

- Provides APIs for message passing between peers.

- Supports unicast, multicast, and broadcast.

4. Replication & Consistency

- Replicates data across peers for availability.
- Ensures updated data consistency across nodes.

5. Fault Tolerance

- Detects failures and reroutes requests.
- Maintains robustness in dynamic environments.

6. Security

- Provides authentication, encryption, and access control.
- Deals with malicious peers.

Architecture of P2P Middleware

 Three layers (simplified):

1. Application Layer

- P2P applications (file sharing, streaming, gaming, storage).

2. P2P Middleware Layer

- Peer discovery
- Routing overlay
- Data indexing & lookup
- Replication management

3. Network Layer

- Underlying Internet protocols (TCP/IP, UDP).



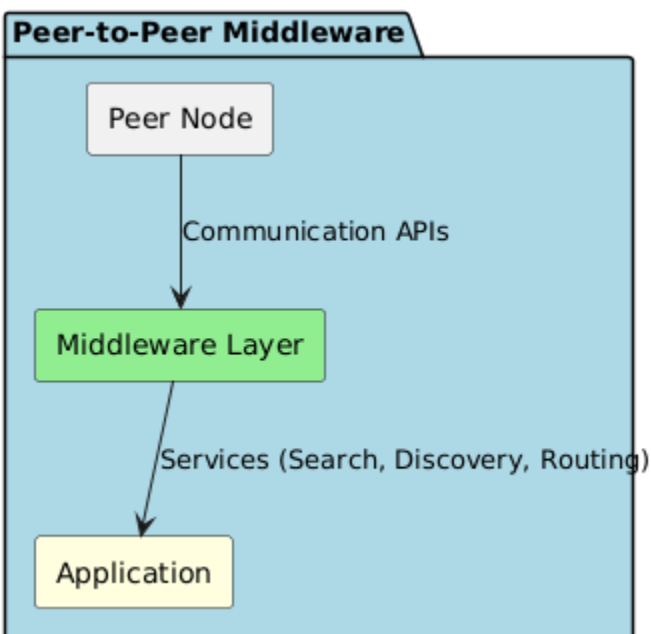
Characteristics

- **Decentralization** – No central authority.
 - **Scalability** – Handles thousands/millions of peers.
 - **Dynamic membership** – Nodes can join/leave anytime.
 - **Self-organization** – System adapts automatically.
 - **Fault-tolerant** – Works even if many peers fail.
-



Examples of P2P Middleware

- **Pastry, Tapestry** (Routing overlays)
- **Chord, CAN, Kademlia** (DHT-based systems)
- **OceanStore, Squirrel** (Application-level middleware)



Routing Overlays (P2P)

What is an overlay?

- An **overlay network** is a **virtual** network formed by links between application nodes (**peers**) on top of the Internet (the **underlay**).
- Peers connect with each other using **logical links** (TCP/UDP) that may traverse many real Internet hops.
- Goal: **find data/peers efficiently** and **route** requests among them, even as nodes join/leave.

Why overlays?

- Internet routing doesn't know about your **keys/files**. Overlays add:
 - **Naming & location** (where is key X?)
 - **Scalable lookup** (fast find even with many nodes)
 - **Resilience** to churn (peers coming/going)

Two big families

(i) Unstructured overlays

- Topology: “random” graph; no rule where data lives.
- **Search:**
 - **Flooding** (send to neighbors with TTL)
 - **Random walks** (probe a few paths)
 - **Super-peer** (powerful nodes index local neighborhoods)
- **Pros:** Simple; good when **queries are fuzzy** (keyword search) and data is **popular**.
- **Cons:** **Inefficient** for rare items; many messages; no guarantees.

(ii) Structured overlays (DHTs)

- Use a **Distributed Hash Table (DHT)**: a **key space** and rules mapping **key** $\rightarrow$ **node**.
- Nodes and keys get **IDs** (e.g., hash of IP or filename).
- **Lookup** in **$O(\log N)$** hops with small routing state.
- **Pros**: Scalable, predictable performance, guaranteed lookup if connected.
- **Cons**: More protocol complexity (join/leave maintenance).

Core DHT ideas (shared across Pastry/Tapestry/Chord/Kademlia)

- **ID Space**: Typically large (e.g., 160-bit).
 - Assign each **node** an ID; each **object (key)** an ID (hash of name).
- **Key placement**: Each key is stored at the node “**responsible**” for it (e.g., the first node clockwise $\geq$ key on a ring).
- **Routing state**: Each node keeps a **small table** of carefully chosen neighbors (fingers, prefix entries, k-buckets, etc.).
- **Routing rule**: Forward a message to a neighbor that **gets closer** to the target key by some metric (ring distance, prefix match, XOR).
- **Replication**: Store copies at a small set of neighbors (e.g., successor list or k closest nodes) for fault tolerance.
- **Caching**: Intermediate nodes may cache popular key $\rightarrow$ node mappings.

Famous structured overlays (very likely to be asked)

- **Chord** (ring + finger table)
 - Metric: **clockwise ring distance** (mod 2^m).
 - State: **Finger table** with $\sim \log N$ entries.
 - Lookup: route to successor of key in **$O(\log N)$** .

- Fault tolerance: **successor list**, stabilization protocol.
- **Pastry / Tapestry** (prefix-based routing)
 - Metric: **prefix match** in base-b representation of IDs.
 - State: **Routing table** (rows by prefix length) + **leaf set** (closest IDs) + neighborhood set.
 - Lookup: each hop **extends the shared prefix** with the target ID → **$O(\log_b N)$** hops.
 - Proximity: try to pick **low-latency neighbors** (proximity neighbor selection).
- **Kademlia** (XOR metric)
 - Metric: **XOR distance** between IDs.
 - State: **k-buckets** by distance ranges; good churn handling.
 - Lookup: parallel, iterative queries toward decreasing XOR distance.

Maintenance under churn (joining/leaving/failures)

- **Join:**
 - Contact any known node (bootstrap).
 - **Locate** responsible region for your ID.
 - **Copy** relevant keys; **populate** routing table/leaf set/fingers.
- **Leave/Crash:**
 - Detect via **timeouts/heartbeats**.
 - **Repair:** replace dead neighbors; update successor/leaf sets; redistribute keys from replicas.
- **Stabilization:**
 - Periodic tasks to **fix pointers**, **refresh entries**, and **rebalance** state.

Optimizations you can mention

- **Virtual nodes (vnodes)**: each physical peer holds multiple IDs → better **load balance**.
- **Proximity neighbor selection**: pick neighbors with lower RTT when choices exist.
- **Replication strategies**: store k copies near the key; **quorums** if strong consistency required.
- **Security hardening**: redundant paths, entry validation to mitigate **Sybil/Eclipse** attacks.

Performance terms (quick bullets)

- **Lookup complexity**: $O(\log N)$ (structured) vs potentially **explosive** messages (unstructured flooding).
- **Path stretch**: overlay hops vs actual Internet shortest path (proximity-aware choices reduce it).
- **Maintenance overhead**: background traffic to keep tables accurate during churn.

When to choose what?

- **Unstructured**: keyword search, approximate matches, high content popularity; tolerate inefficiency.
- **Structured (DHT)**: exact-match lookups, massive scale, predictable performance (e.g., key→value stores, object location).

Small worked example (ring DHT)

- Node IDs on a circle: $0 \dots 2^m - 1$.
- Keys are points on the same circle.
- **Responsible node** = first node **clockwise** from the key.
- Routing: each hop chooses a neighbor **closer clockwise** to the key; in $\sim \log N$ hops you reach the responsible node.

Overlay Case Studies

1. Pastry

- **Definition:** Pastry is a *structured peer-to-peer routing overlay* that provides a scalable, self-organizing, and fault-tolerant infrastructure.
- **Key Features:**
 - Uses **prefix-based routing**. Each node has a **NodeID** (random, 128-bit).
 - Message is routed to the node whose NodeID is numerically closest to the key.
 - Each node maintains:
 - **Routing Table** (log N entries)
 - **Leaf Set** (L/2 closest smaller IDs and L/2 closest larger IDs)
 - **Neighborhood Set** (physically close nodes).
 - Routing complexity: **$O(\log N)$** hops.
 - Supports *load balancing* and *fault tolerance*.

2. Tapestry

- **Definition:** Tapestry is a **decentralized, scalable, fault-tolerant** peer-to-peer overlay, designed for **object location and routing**.
- **Key Features:**
 - Inspired by the **Plaxton Mesh** data structure.
 - Uses **digit-by-digit routing** (like Pastry).
 - Every object has a globally unique identifier (GUID).
 - Supports *surrogate routing* → routes requests to the nearest copy of an object.
 - Ensures **low latency** and **location-aware routing**.

- Complexity: **$O(\log N)$** hops.

Application Case Studies

1. Squirrel

- **Definition:** A peer-to-peer web cache system built on Pastry.
- **Objective:** Reduce **web proxy server bottlenecks** by caching data across nodes.
- **Working:**
 - Each node acts as a web proxy for its local users.
 - A web object (e.g., HTML page) is assigned a **key (hash of URL)**.
 - Pastry routes requests to the node responsible for that key.
 - That node either stores the cached copy or retrieves it from the Internet.
- **Advantages:**
 - Efficient load balancing.
 - Reduces latency and bandwidth usage.
 - Fully distributed (no single point of failure).

2. OceanStore

- **Definition:** A global-scale, persistent storage system built on peer-to-peer architecture.
- **Objective:** Provide **secure, reliable, and ubiquitous storage** across the Internet.
- **Key Features:**
 - Data is **permanently stored**, even if individual servers fail.
 - Uses **erasure coding** for redundancy (splits data into fragments stored at different sites).

- Provides strong **security guarantees** (encryption, authentication).
- Built over a **Tapestry routing substrate**.
- Applications: Long-term archival storage, distributed file sharing.

Comparison Table (Detailed)

Feature/Aspect	Pastry	Tapestry	Squirrel	OceanStore
Type	Routing overlay	Routing overlay	Application (web cache)	Application (global storage)
Main Purpose	Efficient routing & location of objects	Object location, nearest replica routing	Distributed web caching	Persistent, secure global storage
Routing Method	Prefix-based routing	Plaxton Mesh, digit-by-digit	Uses Pastry for routing	Uses Tapestry for routing
Routing Complexity	$O(\log N)$ hops	$O(\log N)$ hops	$O(\log N)$ hops (via Pastry)	$O(\log N)$ hops (via Tapestry)
Data Placement	Closest NodeID to key	Surrogate routing (nearest replica)	Hash(URL) → responsible node	Erasure-coded fragments across nodes
Fault Tolerance	Leaf set & redundancy	Multiple surrogate routes	Replication of cached data	Erasure coding + replication

Load Balancing	Randomized NodeIDs ensure balance	Location-aware routing distributes load	Distributed caching reduces hotspot	Replication spreads load
Scalability	High (millions of nodes)	High (millions of nodes)	Scales with users	Global-scale (Internet-wide)
Applications	General P2P substrate	General P2P substrate	Web caching (reduces bandwidth & latency)	Long-term storage, backup, sharing
Strengths	Simplicity, scalable routing	Location-aware, nearest replica	Efficient, decentralized web cache	Reliable, secure, persistent storage
Weaknesses	Not location-aware	Slightly more complex than Pastry	Depends on Pastry, only web caching	Complex design, heavy resource need

Time and Global States – Introduction

♦ Why is Time Important in Distributed Systems?

- In centralized systems, there is **one system clock**, so events are naturally ordered.
- In distributed systems:
 - Multiple machines, each with its own local clock.
 - **No single global clock** → events may appear in different orders on different machines.

- This makes coordination, ordering of events, and debugging **very challenging**.

◆ **Problems due to Lack of Global Time:**

1. **Event Ordering Issues**

- Example: A message is sent at 10:00 on Machine A but received at 9:59 on Machine B (because of unsynchronized clocks).
- This creates confusion in determining "cause and effect."

2. **Global State Capture**

- To analyze or debug, we need a **consistent snapshot** of the system state.
- Without a common time, it's hard to decide which events happened before/after.

3. **Coordination & Agreement**

- Algorithms like **distributed mutual exclusion, consensus, leader election** need a logical way to order events across nodes.

◆ **Key Goals in Time & Global States:**

● **Provide a notion of time** that works in distributed systems:

- Physical synchronization (close to real clock).
- Logical ordering (causality of events).

● **Define Global State:**

A consistent snapshot of the distributed system → useful for:

- **Checkpointing & recovery**
- **Distributed debugging**
- **Deadlock detection**
- **Consistent backups**

♦ Real vs. Logical Time:

- **Real (Physical) Time:**

- Based on actual clocks (NTP, GPS synchronization).
- Useful for approximating real-world time.

- **Logical Time:**

- Ignores actual clock values.
- Instead, defines time based on **ordering of events** ("happened-before" relationship).
- Used in algorithms like **Lamport clocks, vector clocks**.

Clocks in Distributed Systems

In a **distributed system**, there is no single global clock. Each process runs on a separate machine with its own **local clock** (usually tied to hardware).

But these clocks **drift** and are **not perfectly synchronized**, leading to problems in ordering events and coordinating actions.

Types of Clocks

1. Physical Clocks

- Based on actual time (wall-clock).
- Each computer has a **hardware clock (quartz crystal oscillator)**.
- **Problem:**
 - Clocks drift → one clock may run faster/slower.
 - Leads to **clock skew** (difference in local times).
- Need **synchronization protocols** to keep them close.

Algorithms:

1. Cristian's Algorithm

- One process acts as a time server.
- Steps:
 1. Client sends request to server.
 2. Server replies with its current time.
 3. Client sets its clock = Server time + (Round Trip Delay / 2).

$$T_{\text{client}} = T_{\text{server}} + (T_{\text{round}} / 2)$$

✓ Example:

- Client sends request at 10 ms.
- Server replies with 100 ms.
- Round trip = 20 ms → Delay = 10 ms.
- Client sets clock = 100 + 10 = 110 ms.

Berkeley Algorithm

- Master process polls others.
- Collects all clocks, averages them, and instructs each process to adjust.
- Works even if no accurate external source exists.

2. Logical Clocks

- Do not measure real time.
- Measure **event ordering** (happens-before relationship).

- Ensure all processes agree on a consistent order of events.
- Examples:
 - **Lamport's Logical Clock**

Rules (Lamport Algorithm):

1. Each process P has a counter LC .
2. Before each event at P : $LC = LC + 1$.
3. When sending a message: send (msg, LC) .
4. On receiving (msg, t) at Q :
 - $LC_Q = \max(LC_Q, t) + 1$
 -
 -

Vector Clocks

- Overcome Lamport's limitation.
- Each process maintains a vector of size N (for N processes).

Rules:

1. Initialize $VC[i]=0$ for all.
2. Before local event at P_i : $VC[i] = VC[i] + 1$.
3. On sending message: attach full vector.
4. On receiving message:

Example with 2 processes:

- P_1 : Event $\rightarrow VC_1 = [1,0]$.
- P_1 sends msg to P_2 .

- P2 receives: $VC2 = [1, 1]$.

This way → we can detect concurrency:

- If $VC(x) < VC(y) \rightarrow x$ happened before y .
- If neither $< \rightarrow$ events are concurrent.

Feature	Physical Clocks	Lamport Clocks	Vector Clocks
Basis	Real hardware time	Logical event counter	Vector of counters
Accuracy	Real-time but drifts	No real-time, only order	Causal + concurrency detection
Algorithm	Cristian's, Berkeley	Lamport's Algorithm	Vector Clock Algorithm
Complexity	Simple but needs sync	Simple	More complex ($O(n)$ storage & communication)
Detect concurrency	✗ No	✗ No	✓ Yes
Use cases	Time sync across nodes	Ordering of events	Causality in distributed debugging, snapshot algorithms

Clock Synchronization

A. External Synchronization

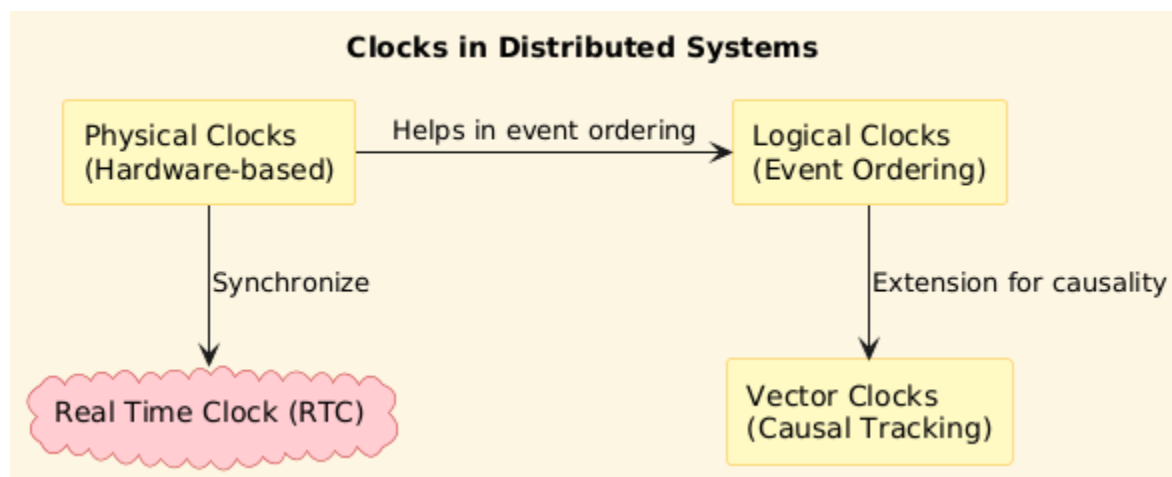
- Local clocks are synchronized with a **real external source** (e.g., UTC via NTP, GPS).
- Bound on clock error:
 $\forall P_i, |C_i(t) - UTC(t)| \leq \delta$

B. Internal Synchronization

- Local clocks in the distributed system are synchronized **with each other**.
- Bound on clock skew:
 $\forall P_i, P_j, |C_i(t) - C_j(t)| \leq \epsilon$

Problems Due to Unsynchronized Clocks

1. Incorrect ordering of events.
2. Failure in coordination (e.g., booking systems, banking).
3. Difficulty in debugging distributed systems.
4. Security issues (expired certificates, authentication tokens).



Events and Process States

In distributed systems, computation is modeled as a set of **processes** that execute **events**. These events represent actions like sending/receiving messages or changing variables. To reason about system behavior, we need to define **events** and **process states**.

1. Events

- An **event** is a single step of execution in a process.
- Types of events:
 1. **Internal Event** → Occurs inside a process (e.g., updating a variable, computing).
 2. **Send Event** → Sending a message to another process.
 3. **Receive Event** → Receiving a message from another process.

👉 Events are the fundamental units used to define **causality** and **ordering**.

2. Process States

- A **process state** represents the snapshot of a process at a given time.
- It includes:
 - Local variables
 - Program counter
 - Buffer of incoming/outgoing messages
- **State transitions** occur due to events:
 - Internal computation → changes local state
 - Send event → modifies outgoing buffer
 - Receive event → modifies incoming buffer

3. Relation between Events and Process States

- Each process is a sequence of events.
- Between two events, the process is in a **state**.
- **State + Event $\rightarrow$ New State**

Formally:

If process **P** is in state **S** and executes event **e**, it moves to state **S'**.

4. Example

Consider **two processes (P1 and P2)**:

- P1 computes (internal event)
- P1 sends a message to P2
- P2 receives the message

Sequence:

P1: e1 (internal) $\rightarrow$ e2 (send m) ----- $\rightarrow$
P2: e3 (receive m)

e1 changes P1's local variable.

e2 updates P1's outgoing buffer.

e3 updates P2's incoming buffer.

5. Algorithmic View (Modeling Events & States)

Algorithm: Event Execution

1. Start with initial state **S0** of process **P**.
2. Pick next event **e**.

- If internal → update local variable.
 - If send → put message in outgoing buffer.
 - If receive → remove message from incoming buffer.
3. Move to new state **S1**.
 4. Repeat until no events left.

6. Diagram (Process Events and States)

Here's a conceptual diagram:

Events and Process States



Synchronizing Physical Clocks – Introduction

- In a distributed system, each computer/node has its **own physical clock**.
- Due to **clock drift** (hardware imperfections), clocks may run faster/slower.
- Without synchronization → timestamps differ → causes inconsistency (e.g., in transactions, logs).
- Goal: Make sure all clocks are approximately the same (within bounded error).

Techniques for Synchronizing Physical Clocks

1. Cristian's Algorithm

- **Used when one machine has access to a reliable time source** (e.g., NTP server, atomic clock).

- Steps:
 1. Client sends request to time server.
 2. Server replies with its current time.
 3. Client sets its clock as:

$$T = T_{\text{server}} + (\text{Round Trip Time} / 2)$$

- Assumption: Network delay is symmetric.
- Problem: If delay is variable, synchronization is less accurate.

Example:

- Client sends at 10:00:00.000
- Server replies at 12:00:00.500
- Round Trip Time (RTT) = 200 ms
- Adjusted Time = 12:00:00.500 + 100 ms = 12:00:00.600

2. Berkeley's Algorithm

- **Used when no node has an external reference.**
- One node is elected as **master** (time coordinator).
- Steps:
 1. Master polls all machines for their clock values.
 2. Computes **average time** (ignores faulty clocks).
 3. Sends adjustment instructions (e.g., +50 ms or -30 ms).
- **Clocks are adjusted gradually** (to avoid sudden jumps).

Example:

- P1 = 10:00:00.200
- P2 = 10:00:00.400
- P3 = 10:00:00.300
- Avg = 10:00:00.300 → Master tells P1 (+100 ms), P2 (-100 ms), P3 (0 ms).

3. Network Time Protocol (NTP)

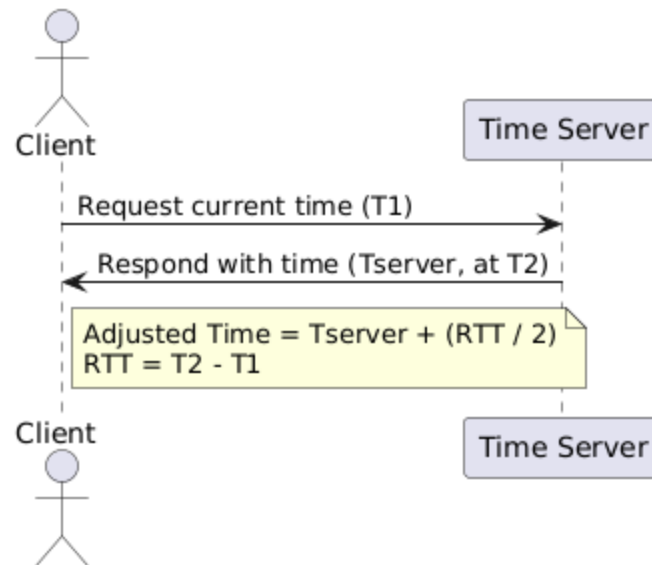
- Internet standard for clock synchronization.
- Uses **hierarchical structure (strata)**:
 - Stratum 0: Atomic clocks, GPS receivers
 - Stratum 1: Directly connected to stratum 0
 - Stratum 2,3,... synchronize from above layers
- Algorithm:
 - Exchanges multiple messages between client & server.
 - Estimates **offset (θ)** and **delay (δ)**:

$$\theta = ((T2 - T1) + (T3 - T4)) / 2$$
$$\delta = (T4 - T1) - (T3 - T2)$$

where:

- T1 = client send time
- T2 = server receive time
- T3 = server reply time
- T4 = client receive time

Cristian's Algorithm (Clock Synchronization)



Logical Time and Logical Clocks

1. Introduction

- In distributed systems, there is **no global clock** shared by all processes.
- Processes need a way to **order events** (cause-effect relationship).
- **Logical clocks** are used to assign timestamps to events such that ordering is preserved.

2. Lamport's Logical Clock

Rules:

1. Each process P_i maintains a logical clock L_i .
2. **Increment rule:** Before executing an event, $L_i = L_i + 1$.
3. **Send rule:** When a message is sent, timestamp $T = L_i$ is attached.

4. **Receive rule:** When P_j receives a message with timestamp T , it sets:
- $$L_j = \max(L_j, T) + 1.$$

3. Example

Suppose we have **2 processes** P1 and P2:

- P1 events: a, b, send(m), d
- P2 events: e, receive(m), f

Steps:

- P1: L_1 increases with each event. When it sends m after b, timestamp is attached.
- P2: When it receives m , it updates its clock as per rule.

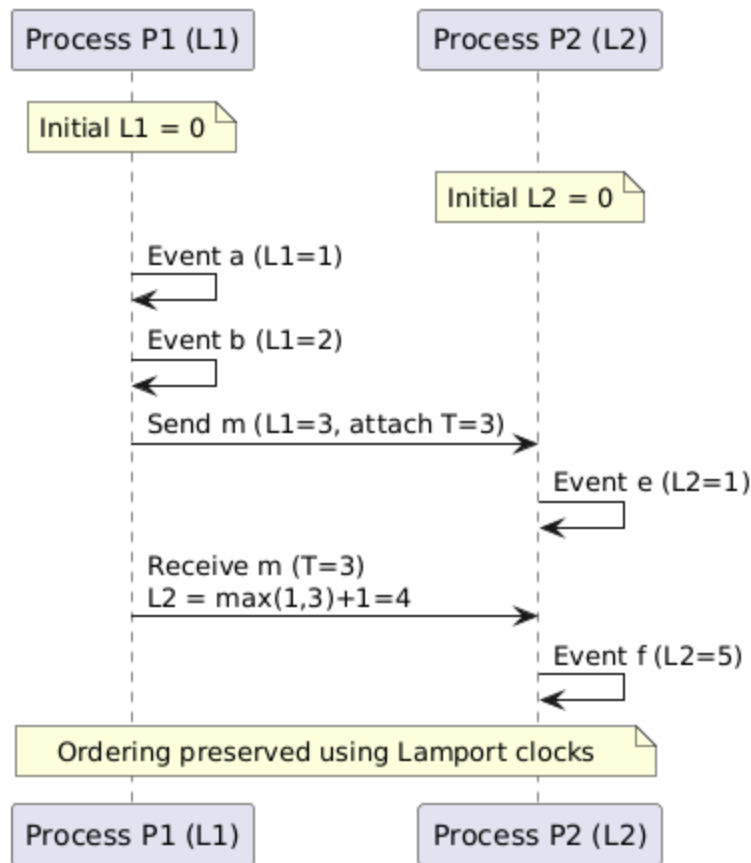
4. Vector Clocks (Improved Version)

- Each process maintains an **array of clocks** $[C_1, C_2, \dots, C_n]$.
- Captures **causal relationships** better than Lamport.

Rules:

1. Before executing event at P_i : $V_i[i] = V_i[i] + 1$.
2. On sending message m , attach vector clock V_i .
3. On receiving message with vector V_j :
 $V_i[k] = \max(V_i[k], V_j[k])$ for all k , then increment $V_i[i]$.

Lamport Logical Clock Example



Global States in Distributed Systems

📌 Definition

- A **global state** of a distributed system is a collection of the **local states of all processes** and the **states of all communication channels** at a particular instant in time.
- Since distributed systems don't share a common clock, capturing a consistent global state is **challenging**.

Importance of Global States

1. **Checkpointing & Recovery** – Helps restore system after failures.

2. **Deadlock Detection** – By examining global snapshots.
3. **Distributed Debugging** – Finding errors across multiple processes.
4. **Monitoring & Consistency** – Ensures correct system behavior.

Techniques for Capturing Global States

1. Chandy-Lamport Snapshot Algorithm

- A classic algorithm to capture consistent global states.

Steps:

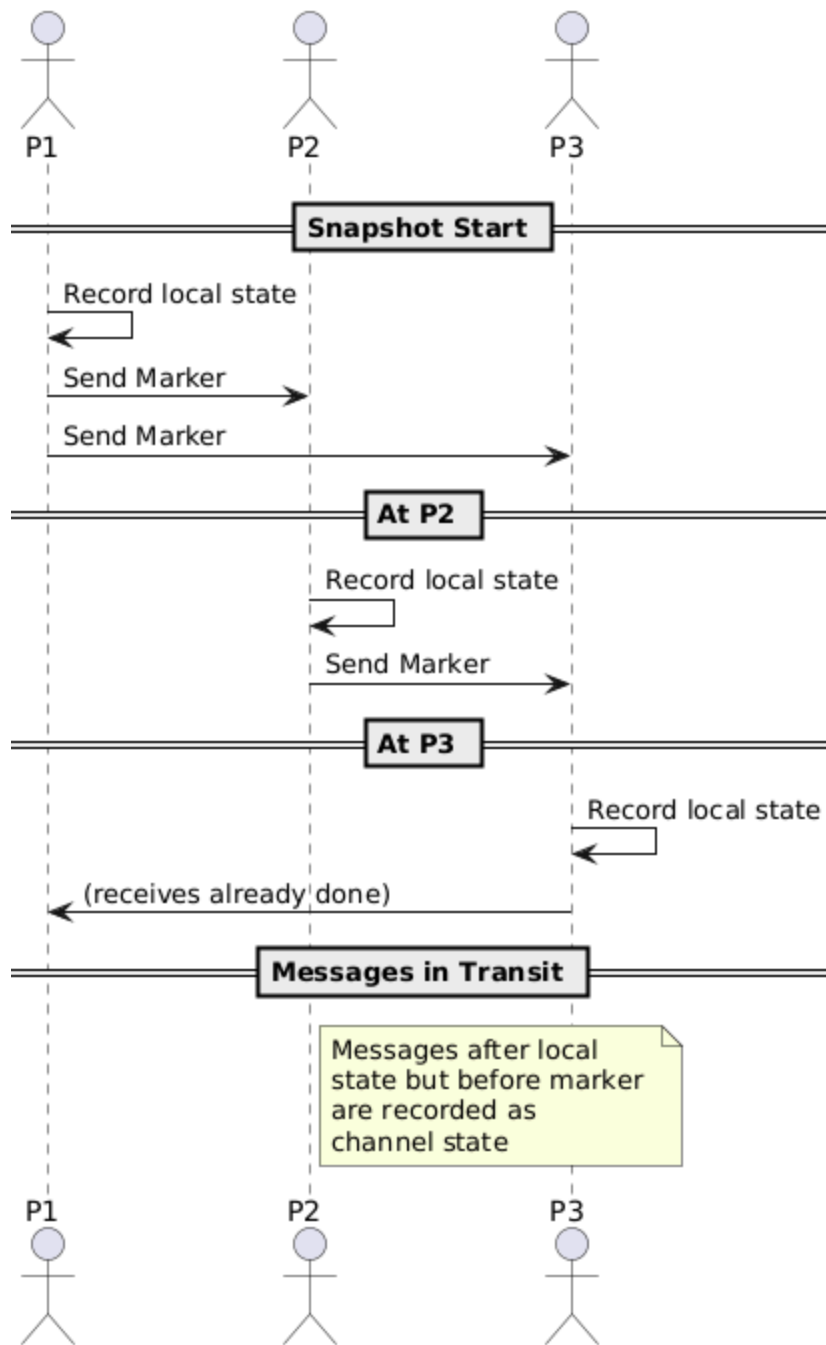
1. Any process can initiate the snapshot by recording its **local state** and sending a **marker message** to all outgoing channels.
2. When a process receives a marker:
 - If it hasn't recorded its local state yet, it records it and sends markers to all its outgoing channels.
 - If it already recorded its state, it records the state of the channel as the set of messages received after recording local state and before receiving marker.
3. When all processes and channels are recorded, the snapshot is complete.

Example

Consider **P1, P2, P3** with message channels.

- P1 starts the snapshot.
- P1 records local state and sends marker → P2 & P3.
- P2, on receiving marker, records local state and forwards marker to P3.
- Messages in transit are also recorded.

Global State Snapshot (Chandy-Lamport)



Distributed Debugging

♦ What is Distributed Debugging?

- Debugging in distributed systems is more complex than in centralized systems because:
 - Multiple processes run on different machines.
 - Communication is asynchronous (messages may arrive late, duplicated, or out of order).
 - There is no single global clock.

♦ Challenges

1. **Non-determinism** – Message delays cause unpredictable execution orders.
2. **Partial Failure** – Some processes may crash while others continue.
3. **Concurrency** – Multiple events can happen at the same time.
4. **Global State Capture** – Hard to capture the system's overall state at once.

♦ Distributed Debugging Approaches

1. Event Logging

- Each process logs local events (send, receive, internal).
- Logs are merged later using timestamps (Lamport clocks / Vector clocks).

2. Global Snapshot Technique

- Uses **Chandy-Lamport Algorithm** to capture global state without halting the system.

3. Replay Debugging

- Record all nondeterministic events, then replay them for debugging.

4. Predicate Detection

- Checks if certain conditions (like deadlock, mutual exclusion violation) ever became true.

♦ Example

Suppose 3 processes (P1, P2, P3) exchange messages:

- P1 sends a request to P2.
- P2 replies after a delay.
- P3 executes independently.

To debug, logs from P1, P2, and P3 are collected and analyzed using logical clocks.

♦ Algorithms Used in Debugging

1. Chandy-Lamport Snapshot Algorithm

- Captures global state without stopping execution.
- Steps:
 1. Initiator sends *marker messages*.
 2. Each process records its local state.
 3. Channel states are recorded as messages “in transit”.
 4. Global state is reconstructed.

2. Vector Clock Analysis

- Helps in ordering events and detecting causality.
- Used to check if a bug (e.g., deadlock) happened before another event.

Coordination and Agreement – Introduction

In Distributed Systems, multiple processes (running on different machines) must often **coordinate and agree** on certain values or actions.

Since there is **no shared global clock** and communication delays are unpredictable, ensuring proper coordination is **challenging**.

Why Coordination & Agreement Are Needed?

- To ensure **consistency** across distributed processes.
- To avoid **conflicts** (e.g., two users booking the same seat).
- To allow systems to **recover from failures** and still function correctly.
- To enable **synchronization** of operations (e.g., database commits, leader election).

♦ Main Problems in Coordination & Agreement

1. Distributed Mutual Exclusion

- Ensuring only one process at a time can access a shared resource.
- Example: Only one process writing to a file.

2. Leader Election

- Selecting a coordinator among distributed processes.
- Example: Choosing a master node in a cluster.

3. Multicast Communication

- Delivering messages to multiple nodes with **reliability and ordering**.
- Example: Sending updates to replicas.

4. Consensus Problem

- All processes must **agree on a single value** despite failures.
- Example: Blockchain nodes agreeing on the next block.

Characteristics of Agreement Protocols

- **Safety:** No two processes decide on different values.
- **Liveness:** All processes eventually reach a decision.
- **Fault Tolerance:** Works even if some processes or messages fail.

Real-World Examples

- **Banking Systems** – prevent double withdrawals.
- **Google Spanner** – uses consensus to ensure globally consistent transactions.
- **Blockchain** – uses consensus algorithms like Proof-of-Work / Raft / Paxos.

Distributed Mutual Exclusion

In **Distributed Systems**, when multiple processes need to access a **shared resource** (e.g., database, file, printer), we must ensure that **only one process enters the critical section (CS) at a time** → this is called **Mutual Exclusion**.

Since there is **no shared memory** or **global clock**, mutual exclusion must be achieved using **message passing**.

Requirements of Distributed Mutual Exclusion

1. **Mutual Exclusion** – At most one process is in the critical section at a time.
2. **Freedom from Deadlock** – No circular waiting.
3. **Freedom from Starvation** – Every request must eventually be granted.
4. **Fairness** – Requests must be granted in order of arrival.
5. **Fault Tolerance** – Should work even with failures.

Approaches to Distributed Mutual Exclusion

1. Centralized Algorithm

- A single coordinator manages CS access.

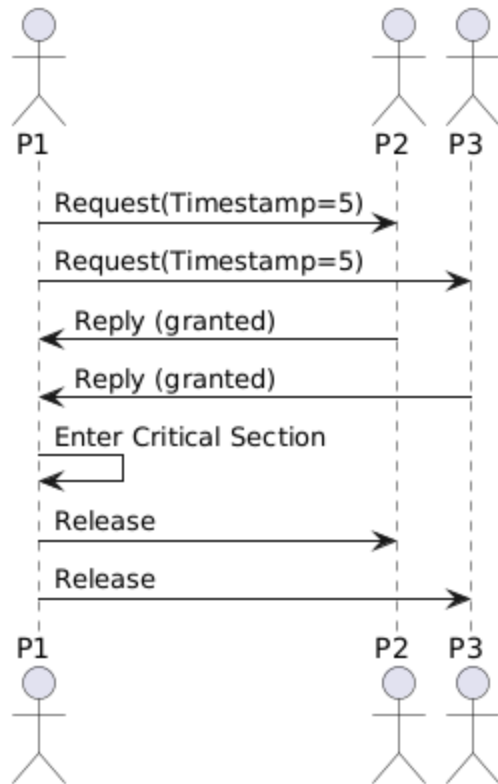
- Steps:
 1. Process sends request to coordinator.
 2. Coordinator grants permission if CS is free.
 3. After using CS, process releases it back to coordinator.
- **Pros:** Simple, fair.
- **Cons:** Coordinator is a bottleneck & single point of failure.

2. Ricart-Agrawala Algorithm (Fully Distributed)

- Based on **timestamped request messages**.
- Steps:
 1. Process sends request (with timestamp) to all other processes.
 2. Each process replies immediately if it's not interested in CS OR if it has a later request.
 3. Process enters CS only after receiving **permission from all others**.
- **Pros:** No single point of failure.
- **Cons:** Needs **$2(N-1)$ messages** per CS entry.

3. Token-Based Algorithm

- A unique **token** is passed among processes.
- Only the process holding the token can enter CS.
- After use, it passes the token to the next requester.
- **Pros:** Only **1 message** needed per entry.
- **Cons:** Token loss → needs recovery mechanism.



Elections in Distributed Systems

Introduction

- In distributed systems, multiple processes (nodes) may run concurrently without a central controller.
- Sometimes, a **coordinator (leader)** is required to manage tasks like:
 - Synchronizing clocks
 - Assigning jobs
 - Managing mutual exclusion
- **Election algorithms** help processes agree on **which node will act as the coordinator**.

Requirements of Election Algorithms

1. **Uniqueness** → Only one leader should be elected.
2. **Agreement** → All nodes must agree on the same leader.
3. **Termination** → The algorithm must end in a finite time.
4. **Fault Tolerance** → If the leader fails, a new one should be elected automatically.

Popular Election Algorithms

1. Bully Algorithm

- Assumes: Each process has a unique ID, and higher IDs have more power.
- **Steps:**
 1. A process detects that the coordinator has failed.
 2. It sends an **ELECTION** message to all processes with higher IDs.
 3. If no higher process responds, it becomes the new coordinator.
 4. If a higher process responds, that process takes over the election.
 5. Finally, the winner sends a **COORDINATOR** message to all others.

✓ **Advantage:** Simple and fast.

✗ **Disadvantage:** Many messages in large systems.

2. Ring Algorithm

- Assumes: Processes are arranged in a logical **ring**.
- **Steps:**
 1. A process notices the coordinator is down.
 2. It sends an **ELECTION** message around the ring containing its ID.
 3. Each process adds its own ID and forwards the message.

4. When the message returns to the initiator, it selects the process with the **highest ID** as the coordinator.
5. A **COORDINATOR** message is sent around the ring to announce the leader.

✓ **Advantage:** Fewer messages than Bully.

✗ **Disadvantage:** Slower if many processes in ring.

Multicast Communication

Introduction

- Multicast communication is a method where a **single process sends a message to a group of processes** in a distributed system.
- Instead of sending multiple copies individually (like unicast), multicast ensures **efficient message delivery** to multiple recipients.
- Used in:
 - Group communication systems
 - Replicated services
 - Collaborative applications (video conferencing, chat systems)

Types of Multicast

1. Unreliable Multicast

- Message may be lost or duplicated.
- Similar to UDP.

2. Reliable Multicast

- Guarantees message delivery to all group members.
- Example: Reliable Multicast Transport Protocol (RMTP).

3. Ordered Multicast

- Ensures messages are delivered in the same order at all processes.
- Variants:
 - **FIFO order**: Messages from a sender are delivered in the order they were sent.
 - **Causal order**: Preserves cause-effect relationships.
 - **Total order**: All processes see the same order for all messages.

Multicast Communication Properties

1. **Atomicity** – Either all processes receive the message or none.
2. **Ordering** – Messages follow a consistent order across processes.
3. **Fault-tolerance** – Must handle node failures.
4. **Efficiency** – Reduce network load compared to multiple unicasts.

Algorithm: Reliable Multicast (Basic)

1. Sender sends message **m** to all members in the group.
2. Each member:
 - Acknowledges receipt to the sender.
 - Stores the message in its buffer.
3. If a process does not receive **m** within a timeout:
 - It requests retransmission.
4. The sender retransmits until all members confirm.

✓ Ensures reliability but does not guarantee ordering.

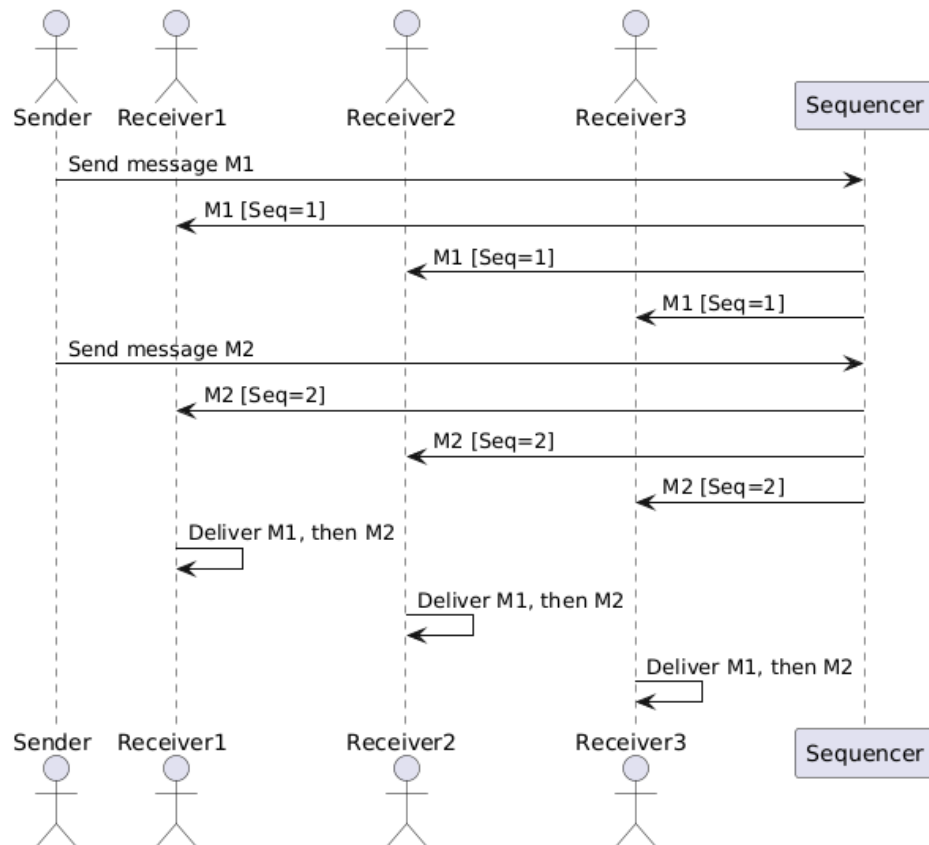
Algorithm: Total Ordered Multicast (Using Sequencer)

1. One process is chosen as the **sequencer**.
2. Steps:
 - A process sends a message to the sequencer.
 - Sequencer assigns a **sequence number**.
 - Sequencer multicasts the message + sequence number.
 - All processes deliver messages in sequence number order.

✓ Ensures all processes see the **same order** of messages.

Example

- Processes: P1, P2, P3
- P1 sends messages M1, M2.
- Sequencer assigns sequence numbers:
 - M1 → Seq 1
 - M2 → Seq 2
- All processes deliver in the order [M1, M2].



Consensus and Related Problems

♦ Introduction

- **Consensus:** The process of making all non-faulty processes in a distributed system agree on a single value.
- Important because processes may fail, messages may be delayed, and yet the system should continue to function consistently.

Applications:

- Distributed databases
- Blockchain
- Leader election

- State machine replication

Properties of Consensus

A consensus algorithm must satisfy:

1. **Termination** – Every non-faulty process must eventually decide.
2. **Agreement** – All non-faulty processes must agree on the same value.
3. **Validity** – The value chosen must be one of the proposed values.

Related Problems

1. Byzantine Generals Problem

- Describes agreement in the presence of malicious or faulty processes.
- Generals must agree on "attack" or "retreat," even if some generals send conflicting orders.

2. Atomic Broadcast

- Ensures all messages are delivered to processes in the same order (used in databases).

3. Fault Tolerance in Consensus

- With **crash faults** → Paxos, Raft.
- With **Byzantine faults** → PBFT (Practical Byzantine Fault Tolerance).

Consensus Algorithms

1. Paxos Algorithm (Crash Fault Tolerant)

- Works with majority agreement.
- Roles:
 - **Proposers** – propose values

- **Acceptors** – decide which proposal to accept
- **Learners** – learn the chosen value

Steps:

1. Proposer sends proposal to acceptors.
2. Acceptors respond with promises (not to accept lower proposals).
3. If majority accepts, the value is chosen.

2. Raft Algorithm (Simpler than Paxos)

- Used in real systems (Etcd, Consul, Kubernetes).
- **Steps:**
 1. **Leader election**
 2. **Log replication**
 3. **Safety:** Leader ensures consistency

3. Byzantine Fault Tolerance (PBFT)

- Works even if some nodes act **maliciously**.
- Requires at least **$3f + 1$ nodes** to tolerate **f Byzantine faults**.
- Used in **Blockchain systems**.

Example – Byzantine Generals

- 3 generals, one is a traitor.
- Loyal generals must agree.
- With **majority voting** → Attack if ≥ 2 vote attack, else retreat.

